

Smart Connected Diagnostics Platform

Engine / Fleet Health Digital Twin · Agentic AI Layer · MCP Tools

A developer's guide to the codebase — what every part is and what it does.

Repository: [stellantis](#) · Project: [stellantis-b2d61](#) · Generated 2026-07-24

1 · What this project is

This is a **Smart Connected Diagnostics Platform**: a set of cooperating backend services and a live web dashboard that continuously monitor vehicles through a **Digital Twin**, detect faults, run **AI-powered diagnostics**, predict failures, recommend fixes, simulate **self-healing** actions, and automatically resolve alerts once telemetry returns to normal.

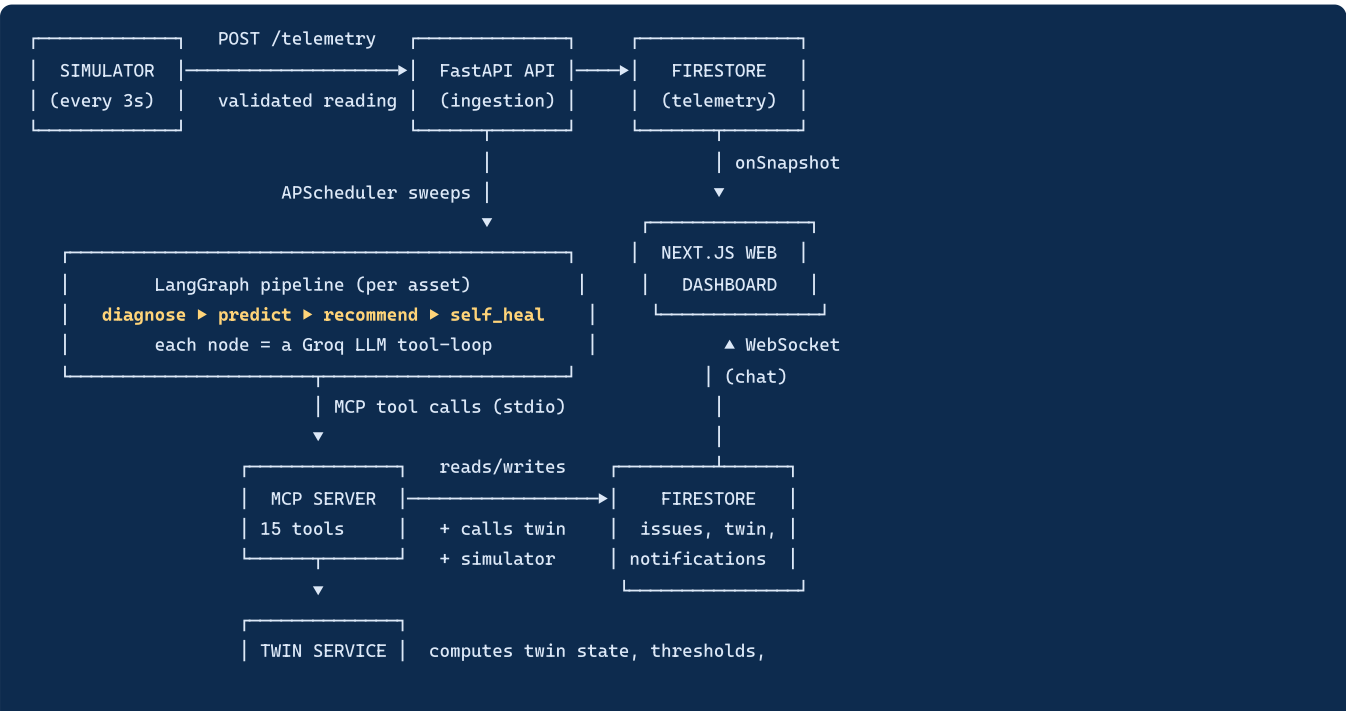
It is built as a **monorepo of independent processes**. A telemetry simulator feeds sensor readings into a FastAPI backend, which stores them in Firestore. A background scheduler periodically runs a **LangGraph** agent pipeline (powered by the **Groq** LLM) that calls tools exposed by an **MCP server**. All results land in Firestore, and the Next.js dashboard renders them in real time through Firestore listeners.

One framing note: the implemented system is a *generalized EV + ICE fleet* platform (it models both electric and combustion vehicles as "assets"), which is a superset of the single-engine spec. Terminology differs accordingly — e.g. "asset" ≈ engine, the `issues` collection holds both alerts and diagnostics, and MCP tools use `module_verb` naming.

Tech stack

Layer	Technology
Frontend	Next.js 16 (App Router, React 19), Tailwind CSS v4, Recharts, Firebase JS SDK
Backend API	FastAPI, Uvicorn, APScheduler, Pydantic v2
Agentic AI	LangGraph (StateGraph), Groq LLM (<code>llama-3.3-70b-versatile</code>), custom ReAct tool-loop
Tool layer	MCP Python SDK (FastMCP server + stdio client session)
ML	scikit-learn (IsolationForest anomaly detection), sentence-transformers (RAG over repair docs), NumPy
Data	Google Cloud Firestore (single shared client)
Realtime	Firestore <code>onSnapshot</code> listeners (dashboard) + WebSocket (chat only)

2 · High-level architecture



There are effectively **four runnable processes** plus the web app: the **simulator**, the **API** (which also hosts the scheduler and, via lifespan, the long-lived MCP client session), the **MCP server** (spawned as a subprocess over stdio), and the **twin service** (an HTTP service the MCP tools call). A shared `libs/common` package holds the schemas and the Firestore client that everyone imports.

3 · Repository layout

Path	Role
<code>libs/common/</code>	Shared Pydantic schemas + the process-wide Firestore client. Imported by every service.
<code>apps/simulator/</code>	Telemetry generator + fault-injection presets. Posts readings to the API.
<code>apps/api/</code>	FastAPI ingestion + REST + chat WebSocket, and the APScheduler background jobs.
<code>services/twin_service/</code>	The Digital Twin engine: EV/ICE models, threshold rules, prediction, simulation.
<code>mcp_server/</code>	MCP server exposing 15 tools (telemetry, twin, diagnostics, actions, knowledge base).
<code>agents/graph/</code>	The LangGraph pipeline, its nodes, the Groq tool-loop, and the MCP client bridge.
<code>apps/web/</code>	Next.js dashboard (fleet overview + per-asset detail, live via Firestore).
<code>firestore.rules / .indexes.json</code>	Firestore security rules (read-only client) and composite index definitions.

4 • Shared foundation — [libs/common/](#)

The contract every other service is built on. Change a field here and it changes everywhere.

[common/schemas.py](#)

Pydantic models that mirror the Firestore documents. Key types:

- `Asset` — one vehicle/engine: `id`, `name`, `vehicle_type` (EV|ICE), `status` (healthy|warning|critical), `risk_score`, `risk_score_numeric`, `last_seen`.
- `TelemetryReading` — a **superset** of EV + ICE sensors; each reading fills only its type's fields. ICE fields: `rpm`, `oil_pressure`, `oil_temp`, `vibration_index`, `fuel_rate`, `air_fuel_ratio_proxy`, `misfire_count`; shared `coolant_temp`; EV fields: `soc`, `soh`, `pack_voltage`, `motor_temp`, `inverter_temp`...
- `Issue` — the unified alert + diagnosis record: `type`, `state` (NEW/OPEN/MONITORING/RESOLVED/ESCALATED), `severity`, `explanation`, `recommendation`, `self_heal_action`, `self_heal_attempts`, `history[]`.
- `TwinSnapshot`, `Notification`, `ChatMessage`, `SustainabilityPeriod` round out the model.

[common/firestore_client.py](#)

A single lazily-created, cached Firestore `Client` shared by all services. Reads credentials from a service-account JSON

(`FIREBASE_SERVICE_ACCOUNT_JSON`) or ADC, plus `FIREBASE_PROJECT_ID`. Also defines the `COLLECTIONS` constants: `assets` · `telemetry` · `twin_snapshots` · `issues` · `notifications` · `chat_sessions` · `sustainability` · `agent_activity`.

[common/vehicle_detection.py](#)

Infers whether an incoming reading is from an EV or ICE vehicle (based on which sensor fields are present), so the API can auto-register a newly seen asset with the correct `vehicle_type`.

5 • Telemetry simulator — [apps/simulator/](#)

[app/main.py](#)

A loop that ticks every `SIMULATOR_TICK_SECONDS` (default 3s), generates the next reading for each asset, and `POST`s it to the API's `/telemetry` endpoint.

[app/state.py](#) & [app/profiles.py](#)

`profiles.py` holds baseline mean values for each sensor per vehicle type. `state.py` generates realistic values via a **mean-reverting random walk** around those baselines — so telemetry drifts naturally instead of jumping.

[app/fault_injection.py](#)

Injectable fault presets used for demos. Each preset **ramps a sensor toward a fault target** and ramps it back to baseline when cleared. ICE presets: `oil_pressure_drop`, `coolant_overheat`, `high_vibration`, `misfire` (EV has battery/motor/inverter presets). This is what lets you inject an "oil leak" and watch the pipeline react.

6 · Backend API + orchestration — apps/api/

app/main.py — application wiring

Creates the FastAPI app. Its **lifespan** is the heart of orchestration: it opens **one long-lived MCP client session** (`mcp_session()`), stashes it on `app.state` , and starts the scheduler with it. CORS is opened for the dashboard at `localhost:3000` . Mounts three routers + a `/health` check.

app/routers/telemetry.py — ingestion

`POST /telemetry` : validates the body against `TelemetryReading` , auto-detects & registers the asset if new, stores the reading under `telemetry/{asset_id}/readings` , and updates the asset's `last_seen` . (Note: the twin is **not** recomputed at ingest — that happens on the diagnostic sweep.)

app/routers/assets.py — REST for assets

Endpoints to list/read assets and trigger fault injection / simulation, backing the dashboard and demo controls.

app/routers/chat.py — "Ask AI" WebSocket

`/ws/chat/{asset_id}` : a WebSocket that drives the **conversational** agent node. It streams back both intermediate **tool-call traces** and the final answer, so the dashboard chat shows the agent's reasoning live.

app/scheduler.py — the beating clock 🕒

Three independent APScheduler jobs, all sharing the single MCP session:

- **Diagnostic sweep** (every 90s): runs the full LangGraph pipeline per asset — the LLM reasoning path (detect → predict → recommend → self-heal).
- **Resolution sweep** (every 20s): a **non-LLM** `diagnostics_resolve_if_recovered` call per open issue — a cheap deterministic state check that drives auto-resolution.
- **Sustainability sweep** (every 300s): records CO₂/energy period data per asset.

Design intent (from the code comments): only spend LLM inference where reasoning helps; recurring state comparisons stay deterministic.

7 · Digital Twin — services/twin_service/

A **stateless** service that computes the current modeled state of an asset on demand from its recent telemetry (it doesn't hold state between calls; persistence lives in Firestore).

app/twins/base.py — the rule engine

`check_thresholds()` compares live metrics against a JSON **fault library** and returns "findings" (which thresholds are breached, and how severely). This is the deterministic monitoring layer under the AI.

app/twins/engine_twin.py (ICE) & ev_twin.py (EV)

Vehicle-type-specific twin models. The ICE engine twin computes derived signals such as a `wear_score` (0–100 from vibration / misfire / oil-pressure). The `factory.py` picks the right twin class for an asset's `vehicle_type`.

app/fault_library/ice_faults.json & ev_faults.json

Declarative threshold rules per subsystem — e.g. `oil_overtemp`, `coolant_overtemp`, `oil_pressure_low`, labeled by subsystem (`lubrication_anomaly`, `cooling_anomaly`, `combustion_anomaly` ...). Adding a new rule is a JSON edit, no code change.

app/models.py & app/main.py

`models.py` defines `TwinState` (metrics + derived values + findings + timestamp). `main.py` exposes the twin over HTTP — including the **what-if simulation** endpoint that recomputes health for hypothetical sensor values.

8 · MCP tool layer — mcp_server/

The standardized tool interface the agents call. `server.py` builds a **FastMCP** server named `diagnostics-platform` and registers five tool modules. Running it (`python -m mcp_server.server`) starts a stdio server the agents connect to.

Module	Tools exposed	What they do
telemetry.py	<code>telemetry_get_latest</code> , <code>telemetry_get_history</code> , <code>telemetry_get_asset_metadata</code> , <code>telemetry_record_sustainability_period</code>	Read sensor readings & asset metadata from Firestore; log sustainability periods.
twin.py	<code>twin_get_state</code> , <code>twin_simulate</code> , <code>twin_predict_failure</code>	Call the twin service for current state, what-if simulation, and failure/RUL prediction (persists a <code>twin_snapshot</code> + risk score).
diagnostics.py	<code>diagnostics_detect_anomaly</code> , <code>diagnostics_get_issue_history</code> , <code>diagnostics_resolve_if_recovered</code> , <code>diagnostics_update_issue</code>	Anomaly detection (thresholds + IsolationForest multivariate ML), issue creation/update, and the auto-resolution state machine.
actions.py	<code>actions_trigger_self_heal</code> , <code>actions_send_notification</code>	Simulate corrective actions (throttle load, eco mode, restart subsystem, schedule service) — effective ones clear the live simulator fault — and write notifications.
knowledge_base.py	<code>kb_search_repair_docs</code>	RAG search over <code>repair_docs.json</code> using sentence-transformer embeddings + cosine similarity, to ground repair recommendations.

Helpers `_twin_client.py` (HTTP to the twin service) and `_firestore_helpers.py` (shared read/write helpers) support the tools.

9 · Agentic AI layer — agents/graph/

A real LangGraph `StateGraph` where each node is itself a small **Groq LLM tool-loop** (not a hardcoded sequence).

graph.py — the pipeline

Builds and compiles the diagnostic graph and runs it per asset:

```
START ▶ diagnose —(anomaly?)→ predict ▶ recommend —(high severity & budget left?)→ self_heal ▶ END
      │(no anomaly)→ END      │(already recommended & unchanged)→ self-heal decision
```

Conditional routing (`_route_after_diagnose` , `_route_after_recommend`) skips expensive re-runs for unchanged open issues and only self-heals **high-severity** issues within a retry budget of 3.

groq_tool_loop.py — the LLM engine

Instantiates the Groq client (`llama-3.3-70b-versatile`) and runs an OpenAI-style **ReAct loop** with `tool_choice="auto"` and retries. Every graph node calls `run_tool_loop` , so each node reasons and calls MCP tools until it has an answer.

mcp_client.py & mcp_tools.py — the bridge

`mcp_client.py` opens the shared stdio `ClientSession` to the MCP server. `mcp_tools.py` translates MCP tool schemas into Groq function-calling definitions and executes the calls the LLM requests.

state.py — shared graph state

`GraphState` is intentionally thin: `asset_id`, `findings`, `issue`, `prediction`, `explanation`, `recommendation`, `self_heal_action`, `self_heal_effective`, `outcome`, `tool_calls[]` . Most "state" is deliberately persisted to Firestore by the tools rather than carried in memory.

nodes/ — the specialist agents

- **diagnostic.py** — detects the anomaly and writes an LLM root-cause explanation (Monitoring + Diagnostic responsibilities).
- **predictive.py** — calls `twin_predict_failure` for failure horizon & risk score.
- **recommend.py** — grounds a repair recommendation via `kb_search_repair_docs` and updates the issue.
- **self_healing.py** — picks & triggers a corrective action; effective actions clear the live fault.
- **conversational.py** — the "Ask AI" agent behind the chat WebSocket.
- **sustainability.py** — computes CO₂/energy periods (beyond the core spec).

activity_log.py

Defines an activity hook that would persist each agent step to the `agent_activity` collection. **NOTE** It exists but is **not currently wired** into the graph nodes (they use a local recorder), so the persistent agent-activity stream is presently unused outside chat.

10 · Dashboard — apps/web/

Next.js 16 App Router app. Realtime is achieved with Firestore `onSnapshot` listeners (no polling); chat uses a WebSocket.

Pages — [src/app/](#)

- **page.tsx** — fleet overview: a grid of asset cards + a notifications aside.
- **assets/[assetId]/page.tsx** → **AssetDetailClient.tsx** — per-asset detail: telemetry charts, issue timeline, sustainability, chat, notifications.
- **layout.tsx** — root layout / shell.

Data hooks — [src/lib/hooks.ts](#)

All realtime reads live here: `useAssets`, `useTelemetryHistory`, `useIssues`, `useNotifications`, `useSustainability` — each a Firestore `onSnapshot` subscription. `firebase.ts` initializes the client SDK; `types.ts` mirrors the backend schemas.

Components — [src/components/](#)

Component	Renders
AssetCard	Per-asset summary: name, EV/ICE type, status badge, risk meter.
AssetTelemetryCharts / MetricLineChart	Recharts line charts of the relevant EV/ICE sensors.
IssueTimeline	Alerts + AI diagnosis: state/severity badges, explanation, recommendation, per-issue history.
NotificationsFeed	Filterable notification stream: alert / resolved / self_heal / escalation / prediction.
ChatPanel	WebSocket "Ask AI" chat with live tool-call traces.
SustainabilityPanel	Actual-vs-baseline CO ₂ bar chart + % savings.
StatusBadge / DeliverablesPanel	Status pills; a summary/marketing grid on the fleet page.

11 · Firestore data model

Collection	Holds	Written by
assets	One doc per vehicle/engine (status, risk score, last seen).	Ingestion (auto-register), diagnostics (health recompute).
telemetry/{id}/readings	Every sensor reading, time-ordered.	Ingestion endpoint.
issues	Unified alerts + diagnostics: state machine, severity, explanation, recommendation, history.	Diagnostics tools / agent nodes.
twin_snapshots	Modeled twin state + predicted failure horizon (written on predict).	<code>twin_predict_failure</code> .
notifications	Dashboard notifications (alert/resolved/self_heal/escalation/prediction).	actions + diagnostics tools.
sustainability/{id}/periods	CO ₂ / energy usage periods.	Sustainability sweep.
agent_activity	(Reserved) per-step agent log.	activity_log hook (not yet wired).
chat_sessions	Chat history.	Chat WebSocket.

Client access is read-only. `firestore.rules` allows browser reads but denies all writes — every write goes through the backend service account (ingestion, MCP tools, scheduler), which bypasses rules. Compound queries (`where asset_id ==` + `orderBy` on a timestamp) require the composite indexes now defined in `firestore.indexes.json`.

12 · End-to-end: the "oil leak" demo flow

1. Dashboard button ▶ API triggers the simulator's `oil_pressure_drop` fault preset.
2. Simulator ramps oil pressure down; readings POST to `/telemetry` ▶ stored in Firestore.
3. Dashboard charts update live (Firestore onSnapshot).
4. Diagnostic sweep (90s) runs the LangGraph pipeline for the asset:
 - `diagnose` – `detect_anomaly` flags low oil pressure ▶ creates an OPEN issue + alert.
 - `predict` – `twin_predict_failure` returns failure horizon + risk score.
 - `recommend` – `kb_search_repair_docs` grounds a repair recommendation.
 - `self_heal` – (if high severity) throttles load / schedules service.
5. Notification written ▶ dashboard NotificationsFeed + IssueTimeline update live.
6. Fault cleared ▶ telemetry normalizes.
7. Resolution sweep (20s) sees findings clear ▶ OPEN ▶ MONITORING ▶ RESOLVED after a stable window.
8. Dashboard shows the issue as Resolved. No human action required.

13 · How to run it

- One shared virtualenv: `pip install -r requirements.txt` (installs the editable `libs/common` too).
- Set `.env`: `FIREBASE_PROJECT_ID`, `FIREBASE_SERVICE_ACCOUNT_JSON`, `GROQ_API_KEY`.
- Run each process: the **API** (`uvicorn app.main:app`, which also spawns the MCP server + scheduler), the **twin service**, and the **simulator**.
- Dashboard: in `apps/web/`, set `.env.local` (Firebase web config + API/WS URLs), then `npm install && npm run dev` → `localhost:3000`.
- Deploy Firestore indexes once: `firebase deploy --only firestore:indexes`.

14 · Implementation status at a glance

Area	Status	Notes
LangGraph + Groq agent pipeline	BUILT	Real StateGraph; each node an LLM tool-loop.
MCP server (15 tools)	BUILT	Different naming than spec; health-index & log_event tools absent.
Diagnostic / Prediction / Recovery agents	BUILT	Solid, with RAG-grounded recommendations.
Auto-resolution lifecycle	BUILT	OPEN → MONITORING → RESOLVED/ESCALATED.

Live dashboard (Firestore realtime)	BUILT	Alerts, diagnosis, notifications, charts, chat.
Supervisory / Monitoring / Notification as agents	PARTIAL	Exist as routing + tool side-effects, not dedicated agents.
Engine Health Index (0–100)	MISSING	Only proxies (wear_score, risk_score_numeric).
Level sensors, failure-injection UI, root-cause graph, what-if sliders	MISSING	Data-model & demo-UI gaps.

This guide reflects the code as of the current branch. Agent-attributed notes above (e.g. the unwired `activity_log`) point to concrete follow-ups if you want to fully match the original spec.